Smart Hospital Management System

MongoDB & PyMongo Implementation • CA2 Practical Project Report

Student: Ojaswi Bhardwaj | **Reg No:** 12319444 | **Roll No:** 37 | **Section:** K3P23SD | **Subject:** CSE494
GitHub: https://github.com/ojaswi1234/mongo_project.git | **Live App:** https://mongo-project-nwjl.onrender.com

Assigned Problem (CA2 Q1): *Design and implement a MongoDB application demonstrating document structure design, validation rules, CRUD operations, PyMongo connection, aggregation queries, and explain query outputs and architectural choices during viva.*

1. Problem Selected & Real-World Clinical Relevance

Hospitals generate continuous streams of clinical data: second-by-second vitals, medications, physician consults, and billing ledgers. In traditional relational databases (SQL), fetching an active patient chart requires 6 to 8 table joins, causing read latency and row-locking bottlenecks during emergencies.

MediTrack leverages MongoDB's document model to solve this. An inpatient's vitals, prescriptions, and bill items are stored inside a single document. The entire medical chart is fetched in an instant, single-read O(1) query without joins. Attending nurses can stream new vitals atomically using `$push` without locking the parent record.

2. Database Architecture & Document Design (Hybrid Model)

The project applies a production **Hybrid Data Modeling Pattern** to balance read speed and storage:

- **Embedded Pattern (patients):** Vitals history, prescriptions, and billing are embedded. They have a 1-to-few bounded relationship (<50 items per stay) and are always queried together with the patient.
- **Referenced Pattern (doctors):** Doctors are stored in a separate collection and referenced via `assigned_doctor_id`. A single physician treats thousands of patients over time (unbounded 1:N). Referencing prevents duplicate physician records and update anomalies.

Sample Inpatient BSON Document (patients collection):

```
{
  "_id": ObjectId("6712ab4e8910cd1234567890"),
  "patient_id": "PAT-1001",                    // Validated format (^PAT-[0-9]{4}$)
  "name": "Robert Miller", "age": 62, "gender": "Male", "blood_group": "A+",
  "department": "Cardiology", "status": "Admitted", "assigned_doctor_id": "DOC-201",
  "vitals_history": [                          // Embedded Time-Series Array
    { "timestamp": ISODate("2026-10-18T08:00:00Z"), "heart_rate": 105, "oxygen_level": 94 }
  ],
  "prescriptions": [                           // Embedded Medications
    { "medicine": "Atorvastatin", "dosage": "20mg", "days": 30 }
  ],
  "billing": { "room_charges": 1600.0, "total_amount": 2330.0, "payment_status": "Paid" }
}
```

3. Validation Rules Implemented ($jsonSchema)

Database-level schema validation prevents corrupt or invalid clinical records from being saved. MongoDB enforces validation rules using `$jsonSchema` with `validationAction: 'error'`:

- **Standardized ID:** `patient_id` must match regex pattern `^PAT-[0-9]{4}$` (e.g., PAT-1001).
- **Sanity Bounds:** `age` must be an integer between 0 and 125. Telemetry heart rate and oxygen levels are restricted to valid physiological ranges.
- **Clinical Enumerations:** `gender` ('Male', 'Female', 'Other') and `status` ('Admitted', 'Under Observation', 'Discharged', 'Archived').

```
// Negative Test: Attempting to insert an invalid document
db.patients.insertOne({ patient_id: 'INVALID_ID', age: 160, gender: 'Alien' })
// Result: pymongo.errors.WriteError: Document failed validation
```

4. PyMongo Implementation & Index Architecture

The Python backend connects to MongoDB using PyMongo with connection pooling and automated failover:

```
from pymongo import MongoClient

# Connection Pooling with timeout failover
client = MongoClient(MONGODB_URI, maxPoolSize=50, serverSelectionTimeoutMS=2500)
db = client['smarthospital']

# 1. Unique Primary Index: Fast O(1) lookups and prevents duplicate IDs
db.patients.create_index([('patient_id', 1)], unique=True)

# 2. Compound Census Index: Speeds up filtered ward census queries
db.patients.create_index([('department', 1), ('status', 1)])
```

5. CRUD Queries Used (Implementation & Operators)

The application performs safe atomic CRUD operations without read-modify-write race conditions:

```
# 1. CREATE: Insert validated patient record
db.patients.insert_one(patient_doc)

# 2. READ: Filtered query with field projection
cardio_patients = db.patients.find(
    {'department': 'Cardiology', 'status': 'Admitted'},
    {'name': 1, 'age': 1, 'blood_group': 1, 'billing.total_amount': 1}
)

# 3. READ ($elemMatch): Matches vitals breached in the exact same reading
critical_cases = db.patients.find({
    'vitals_history': {'$elemMatch': {'heart_rate': {'$gte': 100}, 'oxygen_level': {'$lte': 95}}}
})

# 4. UPDATE ($push & $inc): Atomic vitals append and billing calculation
db.patients.update_one({'patient_id': 'PAT-1001'}, {'$push': {'vitals_history': new_vital}})
db.patients.update_one({'patient_id': 'PAT-1001'}, {'$inc': {'billing.total_amount': 250.0}})

# 5. DELETE: HIPAA-compliant soft delete (archives record instead of hard removal)
db.patients.update_one({'patient_id': 'PAT-1001'}, {'$set': {'status': 'Archived', 'is_archived': True}})
```

6. Aggregation Pipelines Used

MongoDB's Aggregation Framework executes server-side pipelines to process clinical analytics in real time:

```
# Pipeline 1: Department Census & Financial Distribution ($match -> $group -> $sort)
dept_analytics = list(db.patients.aggregate([
    { '$match': { 'status': { '$ne': 'Archived' } } },
    { '$group': {
        '_id': '$department',
        'total_patients': { '$sum': 1 },
        'total_revenue': { '$sum': '$billing.total_amount' },
        'avg_age': { '$avg': '$age' }
    }},
    { '$sort': { 'total_revenue': -1 } }
]))

# Pipeline 2: Doctor Caseload with Cross-Collection Join ($lookup -> $project -> $sort)
doc_workload = list(db.doctors.aggregate([
    { '$lookup': {
        'from': 'patients',
        'localField': 'doctor_id',
        'foreignField': 'assigned_doctor_id',
        'as': 'cases'
    }},
    { '$project': {
        'name': 1, 'department': 1,
        'patient_count': { '$size': '$cases' }
    }},
    { '$sort': { 'patient_count': -1 } }
]))

# Pipeline 3: Critical Vitals Triage Anomaly Alert ($unwind -> $match -> $project)
triage_alerts = list(db.patients.aggregate([
    { '$unwind': '$vitals_history' },
    { '$match': {
        '$or': [
            { 'vitals_history.heart_rate': { '$gte': 100 } },
            { 'vitals_history.oxygen_level': { '$lte': 95 } }
        ]
    }},
    { '$project': { 'patient_id': 1, 'name': 1, 'vitals': '$vitals_history' } }
]))
```

7. Output Obtained from the Queries (Verified Test Results)

Actual outputs obtained from automated test script execution (`scripts/run_demo.py`):

```
1. Schema Validation Rejection Test:
   Insert: patient_id='INVALID_ID', age=160
   Result: [OK] WriteError: Document failed validation rules.

2. Department Analytics Pipeline ($group & $sort):
   • Orthopedics : 1 Patient  | Total Revenue: $3,940.00 | Avg Age: 71.0 yrs
   • Cardiology  : 3 Patients | Total Revenue: $3,835.00 | Avg Age: 48.7 yrs
   • Neurology   : 1 Patient  | Total Revenue: $1,310.00 | Avg Age: 45.0 yrs

3. Doctor Caseload Pipeline ($lookup):
   • Dr. Sarah Jenkins (Cardiology)  : 3 assigned patients
   • Dr. Michael Chen  (Neurology)   : 1 assigned patient

4. Critical Vitals Triage Alert ($unwind & $match):
   • PAT-1005 (Ethan Wright)  : HR = 118 BPM, SpO2 = 93% -> CRITICAL
   • PAT-1001 (Robert Miller) : HR = 105 BPM, SpO2 = 94% -> CRITICAL
   • PAT-1004 (Amina Morales) : HR = 115 BPM, SpO2 = 95% -> URGENT
```

8. Why Particular Queries or Structures Were Selected (Viva Defense)

Key architectural justifications formatted for oral presentation and viva defense:

Q: Why embed vitals and prescriptions instead of using separate collections?

A: High read locality. Doctors and nurses view vitals and medications together on almost every chart lookup. Because vitals are bounded (<50 readings per hospital stay), embedding avoids slow table joins and provides instant O(1) single-document reads.

Q: Why reference doctors instead of embedding them?

A: One doctor treats thousands of patients over years (an unbounded 1-to-many relationship). Embedding doctor profiles would duplicate data across thousands of records and cause update anomalies when doctor fees or details change.

Q: Why use atomic operators ($push, $inc) instead of find-and-save in Python?

A: Multiple healthcare workers modify patient charts concurrently. Atomic operators update data directly inside the MongoDB engine, preventing lost updates and race conditions without requiring heavy multi-document locks.

Q: Why use $elemMatch for telemetry queries instead of dot notation?

A: Standard dot queries match if *any* reading has high heart rate and *any other* reading has low oxygen. `$elemMatch` guarantees that both abnormal thresholds occurred in the **exact same measurement event**, preventing false triage alerts.

9. Project Deliverables Summary

- **Working Application Code:** Clean Flask & PyMongo codebase (`app.py`) • GitHub: https://github.com/ojaswi1234/mongo_project.git
- **Live Cloud Deployment:** Deployed on Render: https://mongo-project-nwjl.onrender.com
- **Automated Test Suite:** Execution script (`python scripts/run_demo.py`) generating `output_demonstration.txt`
- **Project Presentation:** Slide deck summarizing queries and architecture (`SmartHospital_Presentation.pdf`)

10. Academic Evaluation & Submission Sign-Off

Student Declaration: I declare that this project, queries, and report are my original work for CSE494 CA2. **Signature:** _____ **Name:** Ojaswi Bhardwaj (Reg: 12319444) **Date:** 19th October, 2026	**Faculty Evaluator:** Project demonstration, database schema, queries, and viva defense evaluated. **Signature:** _____ **Marks Awarded:** _____ / 30 **Date:** _____